22AIE313 Computer Vision & Image Processing

Lab Sheet 7

Content- Based Image Retrieval (CBIR)

CO3: Develop image recognition algorithms

Aim

To develop an image retrieval algorithm that finds visually similar images from a dataset using handcrafted features.

Tasks

Using your custom dataset, build a system that takes a query image as input and returns the top K most similar images from the dataset.

1. Implement any classical feature extraction method (you can explore and implement techniques that are not taught in the class) and extract features from images.
2. Input a query image and compute similarity between images and the query.
3. Rank the dataset images based on similarity to the query image and display the top k images.
4. Evaluate your system using the metric, Precision@K (Precision@K means how many of top K belong to same category).
5. Test with multiple query images from different categories and report the average Precision@K.

Ananthakrishnans

AM.SC.U4AIE23019

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
!pip install -q kaggle
!kaggle datasets download -d puneet6060/intel-image-classification
!unzip intel-image-classification.zip -d intel_data
```

```
  inflating: intel_data/seg_train/seg_train/street/9770.jpg
  inflating: intel_data/seg_train/seg_train/street/9798.jpg
  inflating: intel_data/seg_train/seg_train/street/9817.jpg
  inflating: intel_data/seg_train/seg_train/street/9843.jpg
  inflating: intel_data/seg_train/seg_train/street/9846.jpg
  inflating: intel_data/seg_train/seg_train/street/9847.jpg
  inflating: intel_data/seg_train/seg_train/street/9863.jpg
  inflating: intel_data/seg_train/seg_train/street/9867.jpg
  inflating: intel_data/seg_train/seg_train/street/9872.jpg
  inflating: intel_data/seg_train/seg_train/street/9875.jpg
  inflating: intel_data/seg_train/seg_train/street/9878.jpg
  inflating: intel_data/seg_train/seg_train/street/9879.jpg
  inflating: intel_data/seg_train/seg_train/street/9885.jpg
  inflating: intel_data/seg_train/seg_train/street/9891.jpg
  inflating: intel_data/seg_train/seg_train/street/9895.jpg
  inflating: intel_data/seg_train/seg_train/street/9903.jpg
  inflating: intel_data/seg_train/seg_train/street/9911.jpg
  inflating: intel_data/seg_train/seg_train/street/992.jpg
  inflating: intel_data/seg_train/seg_train/street/9926.jpg
  inflating: intel_data/seg_train/seg_train/street/9931.jpg
  inflating: intel_data/seg_train/seg_train/street/9934.jpg
  inflating: intel_data/seg_train/seg_train/street/994.jpg
  inflating: intel_data/seg_train/seg_train/street/9953.jpg
  inflating: intel_data/seg_train/seg_train/street/9959.jpg
  inflating: intel_data/seg_train/seg_train/street/9961.jpg
  inflating: intel_data/seg_train/seg_train/street/9967.jpg
  inflating: intel_data/seg_train/seg_train/street/9978.jpg
  inflating: intel_data/seg_train/seg_train/street/9989.jpg
  inflating: intel_data/seg_train/seg_train/street/999.jpg
```

```
Start coding or generate with AI.
```

```
import os
import shutil
import random

base_path = "intel_data/seg_train/seg_train"
new_path = "dataset"

categories = ["buildings", "forest", "glacier", "street"]

os.makedirs(new_path, exist_ok=True)

for cat in categories:
    src = os.path.join(base_path, cat)
    dst = os.path.join(new_path, cat)

    os.makedirs(dst, exist_ok=True)

    images = os.listdir(src)
    selected = random.sample(images, 25)

    for img in selected:
        shutil.copy(os.path.join(src, img), os.path.join(dst, img))

print("100-image dataset ready!")
```

```
100-image dataset ready!
```

```
#q1
import cv2
import numpy as np

def extract_features(image_path):
    image = cv2.imread(image_path)
    image = cv2.resize(image, (256,256))

    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    hist = cv2.calcHist([hsv], [0,1,2],
                        None, [8,8,8],
                        [0,180,0,256,0,256])

    cv2.normalize(hist, hist)

    return hist.flatten()
```

```
def load_dataset(dataset_path):
    features = []
    labels = []
    paths = []

    for category in os.listdir(dataset_path):
```

```
            cat_path = os.path.join(dataset_path, category)

            for file in os.listdir(cat_path):
                img_path = os.path.join(cat_path, file)

                feat = extract_features(img_path)

                features.append(feat)
                labels.append(category)
                paths.append(img_path)

        return np.array(features), labels, paths


    features, labels, paths = load_dataset("dataset")

    print("Dataset Loaded:", len(paths))
```

```
Dataset Loaded: 199
```

```
#q2
def compute_similarity(query_feat, dataset_feats):
    distances = np.linalg.norm(dataset_feats - query_feat, axis=1)
    return distances
```

```
#q3
def retrieve_top_k(query_path, dataset_feats, labels, paths, k=5):
    query_feat = extract_features(query_path)

    distances = compute_similarity(query_feat, dataset_feats)

    indices = np.argsort(distances)
    top_k = indices[:k]

    results = [(paths[i], labels[i], distances[i]) for i in top_k]

    return results
```

```
#q4
def precision_at_k(results, query_label, k):
    correct = sum(1 for _, label, _ in results if label == query_label)
    return correct / k
```

```
import matplotlib.pyplot as plt

def show_results(query_path, results):
    plt.figure(figsize=(12,5))

    query_img = cv2.imread(query_path)
    query_img = cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB)

    plt.subplot(1, len(results)+1, 1)
    plt.imshow(query_img)
    plt.title("Query")
    plt.axis('off')

    for i, (path, label, dist) in enumerate(results):
        img = cv2.imread(path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        plt.subplot(1, len(results)+1, i+2)
        plt.imshow(img)
        plt.title(label)
        plt.axis('off')

    plt.show()
```

```

#q5
query_path = paths[0]
query_label = labels[0]

k = 5

results = retrieve_top_k(query_path, features, labels, paths, k)

show_results(query_path, results)

print("Precision@K:", precision_at_k(results, query_label, k))
```


```
Precision@K: 0.8
```

```
import random

sample_indices = random.sample(range(len(paths)), 10)

precisions = []

for idx in sample_indices:
    q_path = paths[idx]
    q_label = labels[idx]

    results = retrieve_top_k(q_path, features, labels, paths, k=5)

    p = precision_at_k(results, q_label, 5)
    precisions.append(p)

    print(f"{q_label} -> Precision@5:", p)

avg_precision = sum(precisions)/len(precisions)

print("\nAverage Precision@5:", avg_precision)
```

```
buildings -> Precision@5: 0.4
glacier -> Precision@5: 1.0
buildings -> Precision@5: 1.0
street -> Precision@5: 1.0
buildings -> Precision@5: 0.4
glacier -> Precision@5: 0.6
buildings -> Precision@5: 0.4
street -> Precision@5: 0.6
buildings -> Precision@5: 0.2
forest -> Precision@5: 1.0

Average Precision@5: 0.6599999999999999
```